

Guia de Entrega Database Design & Architecture

Metodologia profissional para projetar e documentar bancos de dados — do levantamento de requisitos ao schema pronto para implementação

Baseado no Marco 1 do projeto **NR-12 SaaS** — use como referência para qualquer projeto/cliente futuro deste tipo de entrega.

Sandro Abreu — Engenharia de Dados & Automação

Por que documentar antes de codar

Uma entrega de **Database Design** profissional não é só um arquivo .sql no final. É um pacote de documentos que prova, pro cliente e pra qualquer dev que entrar no projeto depois, que o schema foi *desenhado* — não improvisado. Esse é o diferencial que separa um freelancer comum de uma entrega de consultoria.

- Menos retrabalho: decisões erradas custam muito mais caro depois que o schema já tem dado de produção em cima.
- Confiança do cliente: ele vê o raciocínio, não só o resultado.
- Vira ativo reutilizável: o mesmo pacote de documentos serve de template pro próximo projeto.
- Handoff sem dor: outro dev (ou você mesmo, 6 meses depois) entende o schema sem precisar perguntar nada.

Decisões técnicas não-óbvias (ex: "por que multi-tenant via Row Level Security e não schema por cliente?") ficam registradas em **ADRs (Architecture Decision Records)** — um arquivo curto por decisão, com contexto, decisão e consequências. Nunca se edita um ADR aceito: se a decisão muda, escreve-se um novo ADR que substitui o anterior.

O Pacote de Documentação

Estes são os documentos que toda entrega de Database Design deve conter. Não são burocracia — cada um responde uma pergunta específica que, se não for respondida agora, vira retrabalho depois.

#	Documento	O que contém	Por que importa
1	Requirements Doc	Regras de negócio (requisitos funcionais e não-funcionais), restrições do domínio.	Define o que o schema PRECISA suportar antes de desenhar qualquer tabela.
2	Modelo Conceitual (ERD)	Entidades e relacionamentos, em alto nível, sem detalhe de implementação.	Permite alinhar com o cliente sem exigir conhecimento técnico de SQL.
3	Modelo Lógico	Atributos, tipos, chaves, cardinalidade de cada relacionamento.	É a ponte entre o conceitual e o SQL real.
4	Naming Conventions	Padrão de nomenclatura de tabelas, colunas, constraints e índices.	Consistência: qualquer dev novo entende sem perguntar.
5	Dicionário de Dados	Tabela por tabela, coluna por coluna: tipo, nulidade, default, descrição, regra.	É a referência única de verdade sobre o schema — a primeira coisa que se consulta.
6	Estratégia de Segurança / Acesso	Como o isolamento de dados é garantido (RLS em multi-tenant, ou controle de acesso em geral).	Crítico em qualquer SaaS — mostra que segurança foi desenhada, não improvisada.
7	ADRs	Registro de decisões técnicas não-óbvias, com o porquê.	Evita retrabalho e discussões repetidas; profissionaliza a entrega.

Passo a Passo do Processo

1. Discovery / entrevista de requisitos

Perguntas-chave pro cliente: Quem vai usar o sistema? É multi-tenant (vários clientes isolados) ou single-tenant? Quais são as entidades principais do domínio? Quais regras de negócio variam de cliente pra cliente (ex: prazos, classificações)? Existe algum dado sensível ou regulado?

2. Esboçar o modelo conceitual (ERD)

Desenhar entidades e relacionamentos em alto nível e validar com o cliente ANTES de detalhar tipos e constraints — é muito mais barato corrigir um ERD do que um schema já implementado.

3. Detalhar o modelo lógico

Atributos de cada entidade, tipos de dado, chaves primárias/estrangeiras, cardinalidade exata de cada relacionamento.

4. Definir naming conventions

Decidir uma vez por projeto (singular/plural, snake_case, padrão de FK, como tratar enums) e documentar — depois é só seguir.

5. Escrever o dicionário de dados completo

Cobrir cada tabela e cada coluna sem exceção, incluindo o porquê de decisões não-óbvias (ex: por que um campo é nullable).

6. Desenhar a estratégia de segurança/RLS

Se multi-tenant: políticas de Row Level Security por tabela, função auxiliar de "conta atual", e um plano de teste de isolamento. Se não for multi-tenant: estratégia de controle de acesso equivalente.

7. Registrar os ADRs das decisões não-óbvias

Toda escolha que alguém vai perguntar "por que assim?" no futuro merece um ADR — não é preciso um pra cada detalhe pequeno.

8. Revisão final com o cliente + handoff

Apresentar o pacote completo, capturar ajustes, e só então gerar as migrations SQL reais (DDL + políticas + dados de seed).

Checklist Rápido de Entrega

Use esta lista por projeto — marque conforme for entregando.

- ☐ Requirements doc revisado com o cliente
- ☐ ERD conceitual validado antes de detalhar
- ☐ Modelo lógico completo (tipos, chaves, cardinalidade)
- ☐ Naming conventions documentadas e aplicadas em 100% do schema
- ☐ Dicionário de dados sem nenhuma coluna sem descrição
- ☐ Estratégia de RLS/segurança desenhada e testada manualmente
- ☐ Todas as decisões não-óbvias registradas em ADR
- ☐ Migrations SQL geradas a partir da documentação (não o contrário)

Estimativa de Esforço por Etapa

Etapa	Tempo estimado
Discovery + requirements	2-4h
Modelo conceitual + lógico (ERD)	3-5h
Naming conventions	0.5-1h
Dicionário de dados	3-6h (depende do nº de tabelas)
Estratégia de RLS/segurança	3-5h (multi-tenant) / 1-2h (simples)
ADRs	1-3h
Revisão com cliente + ajustes	1-2h

Como Empacotar Como Gig no Fiverr

Sugestão de estrutura de tiers, no mesmo padrão usado na gig de automação n8n (Basic/Standard/Premium) — **ajustar valores conforme escopo real e mercado antes de publicar.**

Tier	Entregáveis	Preço
Basic	Requirements doc + ERD conceitual + dicionário de dados básico (até ~6 tabelas)	sugestão: a validar
Standard	Tudo do Basic + naming conventions + modelo lógico completo + ADRs das decisões principais	sugestão: a validar
Premium	Tudo do Standard + estratégia de RLS/segurança + script SQL (DDL) pronto pra rodar + 1 rodada de revisão ao vivo	sugestão: a validar

Referência interna: a gig de automação n8n está precificada em Basic \$150 / Standard \$350 / Premium \$700 — use como ponto de partida pra calibrar esta, mas o escopo de Database Design tende a ter menos horas de execução recorrente (não há manutenção contínua como em automação), então o preço pode ser mais baixo ou estruturado como entrega única.

Projeto de Referência

O pacote completo de documentação já foi produzido, seguindo exatamente este processo, em **nr12-saas/docs/database/** e **nr12-saas/docs/adr/** (projeto NR-12 SaaS). Use esses arquivos como amostra/portfólio real ao oferecer esta gig — e também como modelo de estrutura de pastas pra replicar em cada novo projeto:

- docs/database/01-requirements.md
- docs/database/02-data-model.md (ERD em Mermaid)
- docs/database/03-naming-conventions.md
- docs/database/04-data-dictionary.md
- docs/database/05-row-level-security.md
- docs/adr/0001, 0002, 0003...

Quando os próximos marcos do projeto (Backend/API, Automação n8n, Frontend, Pagamentos) forem concluídos, repita este mesmo formato de guia em pastas equivalentes — cada marco do projeto técnico vira sua própria gig documentada.

Gerado como parte do projeto NR-12 SaaS — pasta nr12-saas/banco-de-dados/